

Portabilidad y Orquestación de Docker Compose a Kubernetes

Sesión 5 · CIERRE — y las demos finales del mini-proyecto 🚀

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida

Docente: Jawy Andrés Romero Pinto

MSc · Especialista en ciberseguridad y nube híbrida · LFCE

El camino recorrido

Cuatro sesiones — y todo apuntaba aquí



Fundamentos

Corriste contenedores; entendiste imagen vs contenedor y lo efímero.



Imágenes

Construiste y publicaste TU imagen — con Dockerfile y sin root.



Redes

Conectaste servicios por nombre y aislaste la BD del mundo.



Persistencia

Volúmenes + respaldo restaurado: los datos sobreviven a todo.

Hoy: la app completa como UNA unidad (Compose), la respuesta a «¿quién cuida los contenedores?» (Kubernetes) — y SUS demos finales.

Objetivos de la sesión

Al final de estas 2 horas podrás...

Componer

una aplicación multi-contenedor completa en UN archivo con Docker Compose.

Migrar

tu app entre entornos (sandbox ↔ VM de Azure) con imágenes, compose y respaldo.

Explicar

qué resuelve un orquestador y los conceptos base de Kubernetes (pod, deployment, service).

Demostrar

tu mini-proyecto integrador: la app viva y la restauración en vivo. 🚀

El problema: tu app vive en tu historial de comandos

Seis líneas por servicio, en orden, de memoria... no escala

```
docker network create ean-net
docker volume create datos_db
docker run -d --name db --network ean-net -v datos_db:/var/lib/mysql \
    -e MYSQL_ROOT_PASSWORD=... -e MYSQL_DATABASE=... mysql:8.0
docker run -d --name blog --network ean-net -p 8080:80 \
    -e WORDPRESS_DB_HOST=db -e ... wordpress:6
# ¿y si son 5 servicios? ¿y si lo monta un colega? ¿y en 6 meses?
```

Frágil

Un flag olvidado (el -v de la S4...) y la app se comporta distinto. Ya lo viviste.

No versionable

El historial de la terminal no va en Git. La infraestructura queda en la memoria de alguien.

No portable

Migrar = re-escribir los comandos a mano en el otro host. Error garantizado.

La solución es la misma jugada de la S2: convertir comandos en un ARCHIVO. El Dockerfile de la app completa.

Docker Compose: la app completa en un archivo

docker-compose.yml — el plano de TODO lo que armaste en el curso

```
services:
  db:
    image: mysql:8.0
    volumes:
      - datos_db:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: Ean2026root
      MYSQL_DATABASE: wordpress
  blog:
    image: wordpress:6
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db
    depends_on: [db]
volumes:
  datos_db:
```

Todo lo que aprendiste, declarado:

services

los contenedores (S1)

image

las imágenes versionadas (S2)

la red

implícita: Compose crea una red propia y el DNS por nombre (S3)

ports

la única puerta publicada (S3)

volumes

los datos inmortales (S4)

La app como unidad: up, down, y de nuevo up

Tres comandos para todo el ciclo de vida

docker compose up -d

Levanta TODO: red, volúmenes, contenedores, en orden. Una línea.

docker compose ps

El estado de la aplicación completa.

docker compose logs -f [servicio]

Los logs de toda la app o de un servicio.

docker compose down

Baja y elimina contenedores y red — los VOLÚMENES quedan (S4). 

docker compose down -v

Baja TODO incluyendo volúmenes — solo cuando de verdad quieres borrar los datos. ⚠️

docker compose up -d --scale blog=3

El adelanto de hoy: ¿y si quiero 3 blogs? ...espera al final de la sesión.

El down/up es la prueba de fuego de tu arquitectura: si al volver a subir los datos siguen, hiciste bien la S4.

Migrar la app: la portabilidad, de verdad

Lo que la propuesta del curso prometió — local ↔ nube

QUÉ VIAJA

1. Imágenes → ya están en Docker Hub (S2)
2. docker-compose.yml → un archivo de texto
3. respaldo.sql → los datos (S4)

EL DESTINO

Cualquier host con Docker:
tu VM de Azure (taller S3),
un servidor on-premise, otra nube.

LA MIGRACIÓN

scp compose + respaldo →
docker compose up -d →
restaurar el .sql → LISTO.

Fíjate qué NO viaja: ningún servidor, ninguna configuración artesanal, ningún «pregúntale al que lo instaló».

Ese es el contrato de la nube híbrida que planteamos en la S1: las cargas se mueven a donde convenga — regulación, costos, picos — porque la app ya no está casada con ningún fierro. Es la Fase 2 de tu mini-proyecto.

¿Y quién cuida los contenedores?

El límite de todo lo que hemos hecho: un solo host, cuidado a mano

Si el contenedor muere a las 3 a.m.

Nadie lo reinicia hasta que TÚ llegues. (docker restart eres tú.)

Si el HOST muere

Mueren todos sus contenedores juntos. El volumen era local (S4)...

Si llega el pico de fin de mes

¿Escalar? docker run × 10, a mano, y un balanceador que no tienes.

Si hay que actualizar sin downtime

rm y run — con un rato de servicio caído entre los dos.

La respuesta de la industria: el ORQUESTADOR — un sistema que vigila, repara, escala y actualiza los contenedores por ti, en un clúster de varios hosts.

Orquestar es declarar deseos

Tú dices el QUÉ; el orquestador pelea por el CÓMO, 24/7

Tú declaras (el deseo)

**«Quiero 3 réplicas de login-app:1.1,
con este límite de memoria,
expuestas en el puerto 80,
y que SIEMPRE sea así.»**

Un archivo YAML — como Compose, pero con superpoderes.

El orquestador mantiene (la realidad)

- ¿Murió una réplica? Levanta otra. Solo. A las 3 a.m. (self-healing)
- ¿Murió un HOST entero? Reparte sus contenedores en los demás nodos.
- ¿Pico de demanda? Escala de 3 a 30 con un comando (o solo).
- ¿Versión nueva? Rolling update: reemplaza de a poco, sin downtime.

El cambio mental: de administrar contenedores a declarar estados deseados. De imperativo a declarativo.

Kubernetes: el estándar

Cuatro conceptos — mapeados a lo que YA sabes

POD

La unidad mínima: uno (o pocos) contenedores que viven y mueren juntos.

≈ tu contenedor de siempre, con papeles

DEPLOYMENT

La declaración: qué imagen, cuántas réplicas, cómo actualizar.

≈ tu docker run... pero xN y vigilado

SERVICE

El nombre estable y el balanceo hacia los pods (que van y vienen).

≈ el DNS de tu red de la S3, con balanceador

CLUSTER

Los nodos (hosts) donde K8s reparte los pods.

≈ varios «Play with Docker» actuando como uno

En la nube no instalas Kubernetes: lo alquilas gestionado — AKS en Azure, EKS en AWS (las láminas de la S1). Hoy lo ves EN VIVO en la demo.

Demo guiada: Kubernetes en acción

Lo que vas a ver — con la imagen que construimos en este curso

1

El deployment: login-app con 3 réplicas — un comando, tres pods.

2

El asesinato: matamos un pod a mano... y K8s levanta otro ANTES de que termine de contarlo. (self-healing)

3

El pico de fin de mes: scale a 10 réplicas en un segundo — y de vuelta a 3.

4

El service: una sola puerta estable balanceando a todos los pods — en AKS, con IP pública de Azure.

Tu papel: observar el comportamiento, no memorizar comandos. Kubernetes es un curso entero — hoy siembras el mapa mental.



Eje de seguridad · S5: los secretos, por fin bien guardados

La deuda que arrastramos desde la S2 — saldada

Lo que hicimos en el curso

- Contraseñas en variables de entorno, escritas en el comando (S3, S4).
- Mejor que hornearlas en la imagen (S2 )... pero quedan en el historial de la shell y visibles en docker inspect.
- **Para un lab: aceptable. Para producción: no.**

El estándar profesional

- Compose: secrets desde archivos fuera de Git (.env en .gitignore, secrets: en el YAML).
- Kubernetes: objetos Secret + RBAC (quién puede leerlos).
- Nube: bóvedas gestionadas — Azure Key Vault — con rotación e identidades gestionadas.
- **La regla: el secreto se INYECTA en runtime desde una bóveda; nunca vive en código, imagen ni YAML.**

El hilo completo del curso: imagen sin secretos (S2) → env al run (S3-S4) → bóveda gestionada (el siguiente paso profesional).

El laboratorio de hoy

35 minutos · Play with Docker · y luego, SUS demos

Parte 1 · Tu primer docker-compose.yml (20 min)

Escribe el compose del blog completo (db + blog + volumen), docker compose up -d, y la prueba de fuego: down y up de nuevo — el post sobrevive gracias al volumen (S4).

Parte 2 · Kubernetes guiado (15 min)

Sigue la demo del docente en pantalla (o replica en Killercoda si quieres tocarlo): deployment, self-healing, scale. Tu misión es entender el comportamiento, no los comandos.

Parte 3 · Demos finales del mini-proyecto (30 min)

Cinco minutos por equipo: la app viva, el diagrama, la restauración en vivo y el cierre. El resto del grupo evalúa con la rúbrica.

Entregable de la S5: tu docker-compose.yml funcionando + la demo final de tu mini-proyecto.

Hacia dónde sigue el camino

El curso termina; la práctica apenas empieza

Profundizar Docker

Compose avanzado, multi-stage builds, healthchecks — docs.docker.com y Play with Docker siguen gratis.

Kubernetes en serio

minikube en tu equipo → curso CKAD/CKA (Linux Foundation) → AKS con la capa gratuita de Azure.

Seguridad de contenedores

Trivy y Docker Bench (los viste en vivo), CIS Benchmarks, imágenes distroless, firma de imágenes.

En su entidad

Identificar UNA app candidata a contenerizar: la de ambientes de prueba que más duele montar. Empezar pequeño.

El repositorio del curso

Queda público: guías, decks, talleres y el mini-proyecto — github.com/Hackwy402/gestion_de_contenedores



Las demos finales del mini-proyecto

5 minutos por equipo — el cronómetro no perdona

Minuto 0-1

La app VIVA: muestra la URL funcionando y presenta qué hace.

Minuto 1-2

La arquitectura: tu diagrama — redes, contenedores, volúmenes, la única puerta.

Minuto 2-4

La restauración EN VIVO: mata la BD y tráela de vuelta con tu respaldo.

Minuto 4-5

El cierre: qué fue lo más difícil y qué contenerizarías en tu trabajo.

Rúbrica: 100 pts (ver mini-proyecto.md) + bonus por VM de Azure y no-root. Se evalúa la arquitectura, no la estética de la app.

Lo que te llevas del curso

S1 · El contenedor empaca la app con todo lo suyo: corre idéntica en local y en la nube.

S2 · La imagen es la receta versionada; la infraestructura, un archivo en Git.

S3 · La red decide quién habla con quién; la puerta la abres TÚ — y solo la necesaria.

S4 · Los datos viven en volúmenes y se respaldan con restauración PROBADA.

S5 · Y cuando la escala apriete: se declara el deseo y el orquestador lo mantiene.

Gracias por estas 5 semanas. El repositorio queda abierto — y la nube híbrida, a un docker run de distancia. 🐳